CYBER SHOOTER GAME

DSA PYTHON PROJECT

BY: SAMRA MEHMOOD
ABDULLAH KHAN

PURPOSE OF THE PROJECT

CYBERSHOOTER: A 2D ARCADE SHOOTING GAME

DEVELOPED USING PYTHON & PYGAME

- To create an interactive 2D shooting game where the player controls a jet and shoots down enemies/asteroids.
- Demonstrates Object-Oriented Programming and game design principles using Python and Pygame.
- To apply Data Structures and Algorithms in a real-world scenario.
- Visual idea: Jet icon + target + short tagline — “Shoot. Survive. Score.”

SCOPE & FEATURES

- Player moves jet using arrow keys
- Bullets destroy enemies/asteroids
- Enemies spawn randomly
- Score increases on each kill
- Game Over as soon the player has zero lifes
- Restart option available
- Extra features: Random spawning of life and double bullets for 10 sec

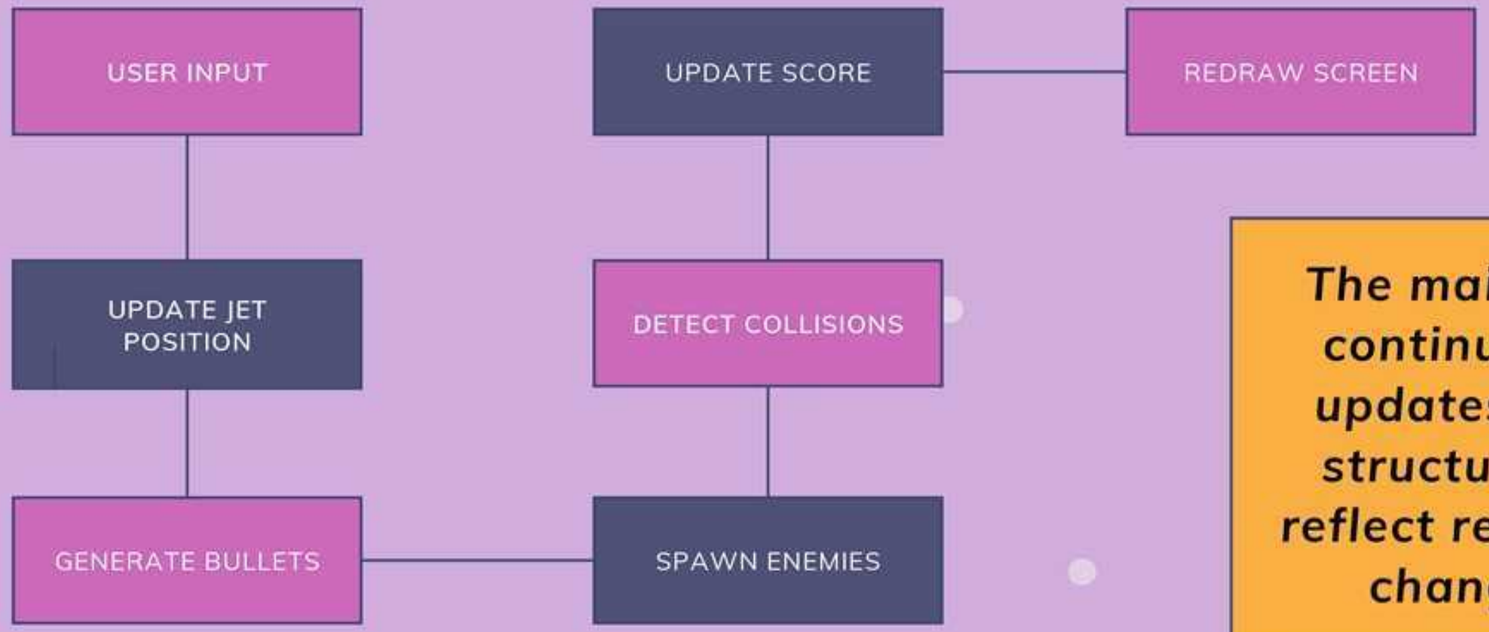
DSA CONCEPTS USED

- Loops: Control continuous game flow and frame updates.
- Queue: Stores elements in FIFO order, useful for managing events or actions that happen sequentially.

- Lists : Store bullets, enemies and positions dynamically.
- Classes & Objects: Jet, Enemy, Bullet — each implemented as an object.
- Stack: Stores elements in LIFO order, useful for undo actions, backtracking or storing temporary game states.

"Each feature in the game demonstrates a key DSA concept in action."

GAME ARCHITECTURE



The main loop continuously updates data structures to reflect real-time changes.

CORE FUNCTIONALITIES

MOVE JET USING KEYBOARD

GAME OVER ON ZERO LIFES

SHOOT BULLETS

ENEMIES/ASTERIODS SPAWN RANDOMLY

DESTROYING ENEMY AND ADDS SCORE

DSA in Action

```python
# Stack (LIFO) using DLL
class Stack:  1 usage
    def __init__(self):
        self._dll = DoublyLinkedList()

    def push(self, value):  3 usages
        return self._dll.append(value)

    def pop(self):
        return self._dll.pop()

    def remove(self, value):  2 usages
        return self._dll.remove_value(value)

    def clear(self):  1 usage
        self._dll.clear()

    def __iter__(self):
        return self._dll.iter_values()

    def __len__(self):
        return len(self._dll)

    def to_list(self):  3 usages
        return self._dll.to_list()
```

```python
# Queue (FIFO) using DLL
class Queue:  4 usages
    def __init__(self):
        self._dll = DoublyLinkedList()

    def enqueue(self, value):  5 usages
        return self._dll.append(value)

    def dequeue(self):
        return self._dll.popleft()

    def remove(self, value):  7 usages
        return self._dll.remove_value(value)

    def clear(self):  5 usages
        self._dll.clear()

    def __iter__(self):
        return self._dll.iter_values()

    def __len__(self):
        return len(self._dll)

    def to_list(self):  9 usages
        return self._dll.to_list()
```

ALGORITHM DESIGN & EFFICIENCY

- Real-time updates require O(n) operations for bullets/enemies each frame.
- Collision checks optimized using bounding boxes instead of pixel-perfect detection.
- Memory-efficient list updates.

"Focus on smooth performance and logical structure, not just visuals."


GAME DEMO

Challenges & Learnings

USING OOP AND BETTER GUI FOR ORGANIZED AND VISUALIZED CODE

DEBUGGING LOGIC FOR COLLISION ACCURACY

HANDLING MULTIPLE LISTS (BULLETS, ENEMIES) SAFELY

MANAGING REAL-TIME UPDATES WITHOUT LAG

FUTURE IMPROVEMENTS

- Use advanced data structures
- Add better sound effects & levels
- Optimize algorithm performance

“Better algorithms, better gameplay.”

THANK YOU!

Cyber Shooter demonstrates how DSA can power interactive applications.

"When data meets design — code becomes a game."